Spring AI Agent Skills

Extensive Roadmap from Basic to Advanced to Expert

A structured learning guide for understanding, designing, implementing, testing, and productionizing Agent Skills with Spring AI.

Covers: concepts, architecture, packaging, tool integration, MCP alignment, governance, and hands-on project progression.

Best for Java/Spring developers building reusable agent capabilities that stay model-agnostic and environment-aware.

Prepared on: 13 March 2026 | **Audience:** Beginner to Expert

How to Use This Guide

- **Beginner layer** — understand what an Agent Skill is, when to use it, and how it differs from plain prompts, tools, and agents.
- **Intermediate layer** — learn packaging, discovery, activation, runtime flow, tool orchestration, testing, and integration in a Spring Boot application.
- **Advanced layer** — design robust skill systems with advisors, memory, RAG, MCP, observability, and safe execution boundaries.
- **Expert layer** — focus on scale, governance, evaluation, multi-agent composition, enterprise rollout, and long-term maintainability.

What Spring AI Says About Agent Skills

Spring's January 2026 Agent Skills article describes skills as modular folders containing a required SKILL.md plus optional scripts, references, and assets. The agent initially loads only the skill name and description, then activates the full instructions on demand when the task matches the skill. This progressive-disclosure pattern helps keep context lean while letting an application expose many capabilities.

In practice, Agent Skills sit beside Spring AI's other core building blocks: tool calling, advisors, chat clients, model portability, and optional MCP-based integration. The result is a composable Java-centric way to package agent know-how without hardcoding every behavior inside one giant system prompt.

Learning Path at a Glance

Stage	Goal	Main Focus	Outcome
Basic	Understand the building blocks	Skill anatomy, terminology, simple examples	Can read and reason about a skill
Intermediate	Build working skill-enabled apps	Spring Boot wiring, tools, prompts, testing	Can create and invoke skills
Advanced	Scale agent behavior safely	Advisors, RAG, memory, MCP, observability	Can build production-grade flows
Expert	Run at enterprise scale	Governance, evaluation, multi-skill systems	Can lead architecture and platform design

Part I — Foundations

1. Core Prerequisites

- Java, Maven or Gradle, Spring Boot fundamentals, dependency injection, configuration properties, profiles, and testing basics.
- HTTP, JSON, REST clients, serialization, and environment variable management.
- LLM basics: prompts, tokens, context windows, temperature, tool calling, hallucination, structured output, and embeddings.
- Agent basics: workflow orchestration vs autonomous behavior, deterministic code vs model-directed actions.

2. Spring AI Fundamentals

- Project purpose: portability and modular design for AI engineering in the Spring ecosystem.
- Core abstractions: ChatClient, ChatModel, Prompt, Messages, Advisors, VectorStore, EmbeddingModel, and ToolCallback.
- Auto-configuration and Boot starters.
- Where agentic systems fit relative to simple chat and RAG applications.

3. What an Agent Skill Is

- Definition: modular folder with instructions and optional executable resources.
- Required file: SKILL.md with YAML frontmatter metadata such as name and description.
- Optional folders: scripts, references, assets, templates, examples, helper files.
- Progressive disclosure model: discovery, activation, execution.
- Why skills help: reusability, lower context overhead, model-agnostic capability packaging, and version control.

4. Distinguishing Skills from Related Concepts

- Skill vs prompt template.
- Skill vs tool: a skill is guidance and packaged capability; a tool is an executable interface exposed to the model.
- Skill vs advisor: advisors intercept and shape requests or responses; skills define reusable operational know-how.
- Skill vs MCP server: MCP standardizes external tools/resources/prompts; a skill can use MCP-backed capabilities but is not the same thing.
- Skill vs workflow engine: workflows define fixed orchestration; skills empower dynamic task execution.

5. Basic Skill Anatomy

- Metadata design: name, description, tags, prerequisites, safety notes, examples, triggers.
- Instruction writing: intent, decision criteria, execution steps, expected outputs, fallback behavior.
- Bundled references: cheat sheets, schemas, architecture notes, domain documentation.
- Bundled scripts: shell, Python, Java CLIs, or wrappers that support repeatable execution.
- Assets and templates: markdown, JSON schemas, starter files, test fixtures.

Part II — Building Skills in Spring AI Applications

6. Skill Runtime Mental Model

- How the agent discovers skills at startup or registration time.
- How only short metadata is loaded first to minimize token usage.
- How full instructions are loaded into context when a task matches the description.
- How the model may then read files or run helper scripts through approved tools.
- How outputs are returned, summarized, or chained into subsequent actions.

7. Project Setup

- Spring Boot app structure for agent features.
- Choosing model providers and starters supported by Spring AI.
- Configuration patterns for API keys, model names, timeouts, and retry policies.
- Local development setup, secrets handling, profiles, and environment segregation.
- Organizing a /skills directory on the filesystem or classpath.

8. Tool Calling as the Execution Backbone

- Spring AI tool calling lifecycle: define tool, expose tool, model requests call, application executes, result returns to model.
- Method-based tools with @Tool and programmatic ToolCallback registration.
- Tool naming, descriptions, and JSON schema clarity.
- Direct return vs model-mediated return.
- Controlling execution with ToolCallingManager and eligibility rules.

9. Wiring Skills into ChatClient

- Combining system prompt, user prompt, tools, advisors, and skill-loading behavior.
- How skill discovery can be represented as a specialized tool or helper service.
- Pattern: expose only skill metadata first, then load full skill text when selected.
- Passing file-reading or shell-execution tools only when required and safe.
- Separating chat orchestration from low-level skill storage and execution services.

10. Building a First Skill

- Example beginner skill ideas: API documentation summarizer, code-review checklist, log triage assistant, SQL explainer, incident note formatter.
- Writing a clean SKILL.md with purpose, when-to-use, step-by-step behavior, and examples.
- Adding a helper script and a reference file.
- Testing the activation path with a simple user request.
- Inspecting how the model chooses the skill and follows the instructions.

11. Recommended Folder Structure

- Single skill per folder.
- Keep SKILL.md concise but operationally clear.
- Prefer small, focused skills over giant all-purpose skills.
- Store references close to the skill they support.

- Separate reusable utilities from skill-specific resources.

12. Prompt Engineering for Skills

- Use explicit triggers and non-triggers.
- State what the model must verify before acting.
- Prefer checklists and stepwise instructions over vague prose.
- Include output format requirements and quality bars.
- Define stop conditions, escalation rules, and uncertainty handling.

Part III — Intermediate Engineering Practices

13. Skill Selection and Routing

- Keyword and semantic matching against skill descriptions.
- Manual routing, model-driven routing, and hybrid routing.
- Conflict resolution when multiple skills seem relevant.
- Ranking by scope, specificity, confidence, and policy fit.
- Fallback when no skill should be loaded.

14. Skills and Advisors Together

- Use advisors for cross-cutting behavior and skills for domain procedures.
- MessageChatMemoryAdvisor for conversation continuity.
- QuestionAnswerAdvisor for retrieval-augmented context.
- Custom advisors for audit metadata, safety filtering, tracing, or response shaping.
- Recursive or chain-style advisor patterns for complex orchestration.

15. Skills with Memory

- When to keep transient conversation memory vs durable task memory.
- Preventing stale memory from overriding live skill instructions.
- Saving outcomes and learned facts carefully.
- Memory partitioning by conversation, user, or task domain.
- Privacy-aware memory design.

16. Skills with RAG

- Using vector search to feed a skill fresh domain context.
- Deciding what belongs in a reference file vs external knowledge store.
- Chunking, metadata filters, citation requirements, and freshness strategy.
- Evaluation of retrieval quality before trusting automated outputs.
- Pattern: retrieval advisor plus domain-specific skill instructions.

17. Skills with Structured Output

- Ask the model to emit JSON, domain records, or typed responses.
- Validate against schemas before downstream execution.
- Use structured output to bridge model reasoning and deterministic services.
- Combine tool results, extracted fields, and final user-facing summaries.

18. Testing and Local Validation

- Unit tests for tool methods and skill services.
- Prompt snapshot tests and regression cases.
- Golden input-output pairs for skill behavior.
- Mocking tool responses and model adapters.
- Failure-path testing: missing files, invalid arguments, permissions denied, timeouts.

19. Skill Observability

- Trace which skill was selected and why.
- Measure latency for activation, file reads, tool calls, and final generation.
- Capture tool inputs/outputs safely.
- Track token usage and context growth.
- Add correlation IDs and structured logs for each agent run.

Part IV — Advanced Agentic Design

20. Multi-Skill Composition

- Sequential composition: one skill prepares artifacts for the next.
- Parallel decomposition with merge or arbitration.
- Coordinator-agent pattern vs direct user-agent pattern.
- Passing intermediate artifacts safely between skills.
- Avoiding instruction collisions between concurrently relevant skills.

21. Dynamic Tool Discovery and Large Toolsets

- As tool counts grow, avoid flooding the model with all definitions.
- Use dynamic tool discovery or search-first approaches so only relevant tools expand into context.
- Pair this with focused skills for better token efficiency and tool precision.
- Design searchable metadata for tools and skills.

22. Skills and MCP

- MCP overview: standardized access to tools, resources, prompts, and utilities.
- How Spring AI supports MCP clients, servers, annotations, and helper utilities.
- How a skill can consume MCP-provided capabilities while remaining provider-agnostic.
- Choosing between direct @Tool methods and MCP-exposed capabilities.
- Security and transport implications when moving from local tools to remote MCP services.

23. Security and Trust Boundaries

- Models do not directly call external APIs; the application executes tool calls.
- Principle of least privilege for file access, shell access, and network access.
- Sandboxing scripts and validating arguments.
- Human approval checkpoints for destructive or high-impact actions.
- Secrets isolation, redaction, and safe logging.

24. Error Handling and Recovery

- Prompt-time recovery: ask for missing input, clarify ambiguity, or refuse unsafe tasks.
- Execution-time recovery: retries, fallback tools, circuit breakers, and graceful degradation.
- State repair after partial completion.
- Idempotency for tasks that can be re-run.
- User-facing transparency when the skill could not finish cleanly.

25. Performance Engineering

- Keep metadata short but discriminative.
- Load large references only when necessary.
- Cache parsed skill manifests and file indexes.
- Limit redundant tool calls and duplicated context.
- Benchmark latency, token consumption, and success rates.

26. Governance and Versioning

- Version skills like code.
- Introduce code review, approval, ownership, and changelogs.
- Define deprecation paths and replacement guidance.
- Track compatibility with model providers, tools, and business policies.
- Use contract tests before publishing skill updates.

Part V — Expert Topics

27. Enterprise Skill Platform Thinking

- Central skill registry or catalog.
- Team ownership, tenancy, and access control.
- Discovery UX for internal developer platforms.
- Policy packs and reusable advisor bundles.
- Metrics dashboards for adoption, quality, cost, and incident rates.

28. Evaluation Frameworks

- Define success metrics: correctness, completeness, latency, cost, safety, and user satisfaction.
- Offline eval sets, shadow traffic, and staged rollout.
- Tool-use accuracy evaluation and false-call analysis.
- Skill-selection precision and recall.
- Regression gates before deployment.

29. Multi-Agent Systems

- Hierarchical sub-agents, planner-worker patterns, and role specialization.
- Assigning skill sets per sub-agent.
- Constrained delegation and resource quotas.
- Conversation contracts between agents.
- When a simple workflow is better than a true autonomous agent.

30. Domain-Specific Skill Design

- Engineering productivity skills: code review, refactoring plans, incident triage.
- Enterprise ops skills: ticket summarization, policy lookup, audit drafting.
- Data skills: query explanation, dashboard narration, ETL diagnosis.
- Customer support skills: response drafting, issue routing, knowledge-grounded troubleshooting.
- Compliance-heavy skills: deterministic checks, evidence capture, and approval flows.

31. Anti-Patterns

- One gigantic skill that tries to solve everything.
- Unclear descriptions that route too often.
- Skills that depend on hidden tribal knowledge.
- Giving shell or file tools without strict controls.
- Mixing policy, orchestration, and business logic into one unreadable instruction file.
- Skipping evaluation because the demo looked good once.

32. Road to Mastery

- Read official Spring AI docs for tools, advisors, MCP, and effective-agent patterns.
- Implement two simple local skills and one skill that uses a tool.
- Add tests, logging, and a small retrieval layer.
- Introduce policy checks and human approval for risky actions.
- Build a shared internal skill library with governance and metrics.

Sample SKILL.md Skeleton

A practical skill should be readable by humans, unambiguous for the model, and easy to review in source control.

```
---
name: incident-triage
description: Analyze application incidents, inspect logs, identify likely causes, and
produce a structured triage summary.
tags:
  - operations
  - logs
  - troubleshooting
safety:
  - Never claim a root cause without evidence.
  - Escalate if required files or logs are missing.
---


# Purpose
Use this skill when a user asks to diagnose an application incident from logs or
metrics.


# Inputs
- Error logs
- Deployment timestamp
- Service name
```

```
- Optional incident context

# Process
1. Confirm scope and time window.
2. Read the provided logs and references.
3. Extract recurring errors, affected components, and timing correlations.
4. Propose likely causes with confidence levels.
5. Recommend next actions and what evidence is still missing.

# Output
Return:
- incident summary
- probable causes
- evidence used
- recommended next actions
- confidence level
```

Sample Spring AI Tool and ChatClient Wiring

```
@Component
class LogTools {

    @Tool(description = "Read recent error logs for a service and time range")
    public String readLogs(String serviceName, String fromTs, String toTs) {
        // fetch logs from your log system
        return "...";
    }
}

@Bean
ChatClient chatClient(ChatModel chatModel, LogTools logTools) {
    return ChatClient.builder(chatModel)
            .defaultTools(logTools)
            .build();
}

// The skill-selection layer decides whether to load the skill text
// and then invokes ChatClient with the relevant instructions + tools.
```

90-Day Study Plan

Phase	Weeks	What to Build	Milestone
Phase 1	1–3	Read Spring AI basics, create one local skill, one @Tool method, simple CLI/chat endpoint.	Understand lifecycle end to end

Phase 2	4–6	Add advisors, memory, and structured output. Create evaluation samples.	Intermediate quality and repeatability
Phase 3	7–9	Add retrieval, logging, policy checks, and human approval for risky operations.	Production-aware architecture
Phase 4	10–13	Introduce multi-skill composition, MCP exploration, governance, and rollout metrics.	Expert-ready platform thinking

Hands-On Project Ideas

- Developer Copilot for internal APIs: a skill library that reads API docs, suggests usage, and generates integration notes.
- Ops Triage Agent: route incidents, read logs with tools, summarize likely causes, and produce a remediation checklist.
- Knowledge-Grounded Support Agent: combine retrieval, skills, and structured output to create support responses with evidence.
- Secure Change Assistant: analyze requested changes, run validation tools, draft implementation plans, and require approval before execution.
- Internal Skill Registry Demo: catalog skills, search by metadata, record usage metrics, and compare success rates.

Checklist for Production Readiness

- Every skill has a clear name, concise description, scope boundary, and owner.
- Every executable tool is permission-scoped and audited.
- Every critical workflow has evaluation cases and regression tests.
- Every skill update follows review and release discipline.
- Every user-facing result can explain evidence, confidence, and next steps.

Official References to Study Next

- Spring AI project overview
- Spring AI Reference: Tool Calling
- Spring AI Reference: Advisors API

- Spring AI Reference: Model Context Protocol (MCP)
- Spring AI Guide: Building Effective Agents
- Spring blog (13 January 2026): Spring AI Agentic Patterns (Part 1): Agent Skills

Final Advice

Master Agent Skills by treating them as disciplined software assets, not prompt scraps. The strongest Spring AI solutions combine small reusable skills, well-described tools, cross-cutting advisors, strong testing, and clear execution boundaries.

Start simple, measure everything, and only increase autonomy when the surrounding controls, observability, and evaluation are strong enough to support it.